

AI & Cloud Computing Viva Questions & Answers

SPPU · TE Computer Engineering · 2019 Course

310253 Artificial Intelligence | 310254(C) Cloud Computing

Theory + Lab Practicals · All 6 Units Each · Complete Coverage

★ = Important / Most Asked ♦ = Practical / Lab Related

Covers: definitions · algorithms · code logic · complexity · applications · output

HOW TO USE THIS DOCUMENT

★ IMPORTANT — Most asked in viva, prioritize these.

♦ PRACTICAL — Code logic, complexity, output, and application are tested.

CODE: lines — actual code you should be able to write/explain on the board.

NOTE: lines — examiner tips and common follow-up points.

Part I covers AI (Units I–VI + Lab Group A/B/C). Part II covers Cloud Computing (Units I–VI + Lab).

PART I — ARTIFICIAL INTELLIGENCE

310253 · All 6 Theory Units + Lab Assignments (Group A, B & C)

UNIT I · INTRODUCTION TO AI

07 Hours

Course Outcomes: CO1, CO4

Q1

★ IMPORTANT

What is Artificial Intelligence (AI)?

→ Artificial Intelligence is the simulation of human intelligence processes by machines, especially computer systems. It involves creating systems that can perform tasks that typically require human intelligence such as reasoning, learning, problem-solving, perception, language understanding, and decision-making.

■ *Key branches include Machine Learning, Natural Language Processing, Computer Vision, Robotics, and Expert Systems.*

Q2

★ IMPORTANT

What are the Foundations of AI?

→ AI is built on multiple disciplines: Mathematics (logic, probability, statistics), Computer Science (algorithms, data structures), Philosophy (logic, reasoning, ethics), Linguistics (natural language), Neuroscience (biological basis of intelligence), Psychology (cognitive science), Economics (decision theory, game theory), and Control Theory.

■ *These interdisciplinary roots make AI a broad and versatile field.*

Q3

★ IMPORTANT

Briefly trace the History of Artificial Intelligence.

→ 1943: McCulloch & Pitts – first mathematical model of a neuron. 1950: Alan Turing proposed the Turing Test. 1956: Dartmouth Conference – the term 'AI' coined by John McCarthy. 1960s–70s: Early programs (ELIZA, SHRDLU). 1980s: Expert Systems boom. 1987: AI Winter (funding cuts). 1997: Deep Blue beats Kasparov. 2012: Deep learning revolution (ImageNet). 2016: AlphaGo beats world champion. 2020s: Large Language Models (GPT, Claude) dominate.

Q4**What is the Turing Test and its significance?**

→ Proposed by Alan Turing in 1950, the Turing Test evaluates a machine's ability to exhibit intelligent behavior indistinguishable from a human. A human evaluator conducts text conversations with both a machine and a human; if the evaluator cannot tell them apart, the machine passes the test. It remains a philosophical benchmark though modern AI surpasses some aspects while failing others.

Q5**★ IMPORTANT****What are the Risks and Benefits of AI?**

→ Benefits: Automation of repetitive tasks, improved accuracy in diagnosis, personalized recommendations, 24x7 availability, data analysis at scale, enhanced safety (autonomous vehicles).

→ Risks: Job displacement, algorithmic bias, privacy invasion, weaponization (autonomous weapons), concentration of power, loss of human oversight, existential risk from misaligned superintelligence.

■ *SPPU expects you to cite real examples — e.g., Kroger using AI and robots in retail as the case study for this unit.*

Q6**★ IMPORTANT****What is an Intelligent Agent?**

→ An Intelligent Agent is any entity that perceives its environment through sensors and acts upon that environment through actuators to achieve its goals. The four key components are: Percepts (input), Actions (output), Goals (objective), Environment (context). Examples: a thermostat (simple reflex agent), a chess program, a self-driving car, a chatbot.

Q7**★ IMPORTANT****Define the PEAS framework for describing AI agents.**

→ PEAS stands for: Performance Measure (criteria for success), Environment (where the agent operates), Actuators (outputs/actions), Sensors (inputs/perceptions). Example for a self-driving car — P: safe arrival, traffic law compliance; E: roads, traffic, weather; A: steering, brakes, horn; S: cameras, GPS, lidar.

■ *PEAS is the standard way to define an agent for exam answers.*

Q8**★ IMPORTANT****What is Rationality in the context of AI agents?**

→ A rational agent selects actions that maximize its expected performance measure given the percept sequence so far and any built-in knowledge. It is NOT omniscient (it doesn't know everything) and NOT clairvoyant (it cannot predict the future). Rationality depends on: the performance measure, prior environment knowledge, the agent's actions, and the percept sequence.

■ *Rationality ≠ perfection. Rational means doing the best given what is known.*

Q9**★ IMPORTANT****What are the different types of Environments in AI?**

- Fully vs Partially Observable: can the agent see complete state?
- Deterministic vs Stochastic: does action always produce same result?
- Episodic vs Sequential: does current decision affect future?
- Static vs Dynamic: does environment change while agent is deliberating?
- Discrete vs Continuous: finite vs infinite percepts/actions
- Single-agent vs Multi-agent: one or many agents interacting?
- Known vs Unknown: are the laws of the environment known?

■ *Real environments are mostly partially observable, stochastic, sequential, dynamic, continuous.*

Q10**★ IMPORTANT****What are the different types of Agent Structures?**

- Simple Reflex Agents: act only on current percept using condition-action rules.
- Model-Based Reflex Agents: maintain internal state of the world.
- Goal-Based Agents: act to achieve specified goals.
- Utility-Based Agents: maximize a utility function for best outcome among alternatives.
- Learning Agents: improve performance over time through experience — consist of a learning element, performance element, critic, and problem generator.

■ *Learning agents are the most powerful and closest to human intelligence.*

Q11**What is the difference between a goal-based and utility-based agent?**

- Goal-based agents have a binary success/failure — they either achieve the goal or not. They cannot handle conflicting goals or trade-offs. Utility-based agents assign a numeric utility (happiness score) to each state and choose actions that maximize expected utility. They handle trade-offs gracefully — e.g., reaching the destination quickly vs safely. Utility functions generalize goals.

Q12**Explain the concept of Learning Agents in detail.**

- A Learning Agent has four components: (1) Performance Element: selects actions (the traditional agent). (2) Learning Element: makes improvements using feedback from the critic. (3) Critic: evaluates the agent's performance against a fixed standard and provides feedback. (4) Problem Generator: suggests exploratory actions that may lead to better future performance. Over time, the agent shifts from random exploration to optimal exploitation.

■ *This is the AI analogy of how humans learn from experience.*

Q13

What is the State of the Art in AI today?

→ Current AI achievements: Large Language Models (GPT-4, Claude) for text generation and reasoning; AlphaFold for protein structure prediction; Stable Diffusion/DALL-E for image generation; self-driving vehicles (Waymo); AlphaStar (StarCraft II); robotic surgery assistants; AI-powered drug discovery; real-time language translation; recommendation systems (Netflix, YouTube); fraud detection in banking.

UNIT II · PROBLEM SOLVING

07 Hours

Course Outcomes: CO2, CO4

Q14

★ IMPORTANT

What is a Problem-Solving Agent?

→ A Problem-Solving Agent is a goal-based agent that plans ahead — it considers a sequence of actions and chooses the one that leads to its goal. It operates in a fully observable, deterministic, discrete, and known environment. The agent formulates the problem, searches for a solution, and executes it.

→ Steps: Goal formulation → Problem formulation → Search → Execution.

Q15

★ IMPORTANT

How is a search problem formally defined?

→ A search problem is defined by 5 components:

→ (1) Initial State: starting configuration.

→ (2) Actions: set of possible actions from any state (Successor function).

→ (3) Transition Model: result of performing action in a state.

→ (4) Goal Test: checks if current state is the goal.

→ (5) Path Cost: numeric cost of each path (sum of step costs).

→ A solution is a sequence of actions from initial to goal state. An optimal solution has the lowest path cost.

Q16

★ IMPORTANT

✦ PRACTICAL

What is Breadth-First Search (BFS)? Explain with algorithm and complexity.

→ BFS explores the search tree level by level, visiting all nodes at depth d before going to depth $d+1$. It uses a FIFO queue (frontier).

→ Algorithm:

1. Enqueue initial state
2. While queue not empty:
 - a. Dequeue node
 - b. If goal, return solution
 - c. Expand node, enqueue unvisited children

→ Properties: Complete: Yes (if branching factor b is finite). Optimal: Yes (if all step costs equal). Time Complexity: $O(b^d)$. Space Complexity: $O(b^d)$ — stores all nodes at current depth.

■ Space is the main weakness of BFS — at depth 12, branching factor 10, BFS needs ~1TB RAM.

Q17

★ IMPORTANT

✦ PRACTICAL

What is Depth-First Search (DFS)? Explain with algorithm and complexity.

→ DFS explores the deepest node first using a LIFO stack (or recursion). It goes as deep as possible before backtracking.

→ Algorithm:

1. Push initial state onto stack
2. While stack not empty:
 - a. Pop node
 - b. If goal, return solution
 - c. Push unvisited children

→ Properties: Complete: No (fails in infinite-depth spaces unless repeated state checking used). Optimal: No. Time: $O(b^m)$ where m is max depth. Space: $O(bm)$ — linear space, major advantage over BFS.

■ DFS can get trapped in infinite loops without visited set.

Q18**What is Depth-Limited Search (DLS)?**

→ DLS is DFS with a pre-specified depth limit l . Nodes at depth l are treated as if they have no successors. This avoids infinite loops in infinite state spaces. Complete: Yes if solution depth $d \leq l$. Optimal: No. Time: $O(b^l)$. Space: $O(bl)$. Drawback: if $l < d$, no solution is found even though one exists.

Q19**★ IMPORTANT****What is Iterative Deepening Depth-First Search (IDDFS)?**

→ IDDFS repeatedly applies DLS with increasing depth limits (0, 1, 2, ...) until a solution is found. It combines BFS's completeness and optimality with DFS's low memory usage.

→ Complexity: Time: $O(b^d)$ — nodes at depth d are generated once, but nodes closer to root are regenerated multiple times (acceptable overhead). Space: $O(bd)$ — linear like DFS.

■ IDDFS is the preferred uninformed search method for large unknown search spaces.

Q20**★ IMPORTANT****What is Uniform Cost Search (UCS)?**

→ UCS expands the node with the lowest cumulative path cost $g(n)$, not simply by level. It uses a priority queue ordered by path cost. Complete: Yes (if step costs $\geq \epsilon > 0$). Optimal: Yes.

→ Time/Space: $O(b^{(1 + \lceil C^*/\epsilon \rceil)})$ where C^* is the optimal solution cost. UCS is Dijkstra's algorithm applied to trees. It's optimal but can be slow if there are many low-cost, non-goal paths.

Q21**★ IMPORTANT****What is Greedy Best-First Search?**

→ Greedy Best-First Search expands the node that appears closest to the goal using a heuristic function $h(n)$ = estimated cost from n to goal. It uses a priority queue ordered by $h(n)$. Complete: No (can loop). Optimal: No (takes locally best but globally suboptimal path). Time/Space: $O(b^m)$ in worst case, but often much better in practice.

■ It's like navigating by compass direction — fast but may not find shortest route.

Q22**★ IMPORTANT****♦ PRACTICAL****What is A* Search? Why is it optimal?**

→ A* combines UCS and Greedy search using $f(n) = g(n) + h(n)$, where $g(n)$ = actual cost from start to n , $h(n)$ = estimated cost from n to goal. It expands nodes in order of $f(n)$.

→ Optimality: A* is optimal if $h(n)$ is ADMISSIBLE (never overestimates) and CONSISTENT ($h(n) \leq \text{cost}(n \rightarrow n') + h(n')$).

→ Complexity: Time/Space: $O(b^d)$ — exponential, but much better in practice with good heuristics. A* is complete and optimal among all best-first searches.

■ Used in GPS, video game pathfinding, robotics, and the practical assignment for any game search problem.

Q23**★ IMPORTANT****What is a Heuristic Function? What makes a good heuristic?**

→ A heuristic function $h(n)$ estimates the cost from node n to the goal. Properties of a good heuristic:

→ Admissibility: $h(n) \leq \text{actual cost}$ (never overestimates) — ensures A* optimality.

→ Consistency (Monotonicity): $h(n) \leq c(n, a, n') + h(n')$ — triangle inequality, ensures A* never re-expands nodes.

→ Informativeness: higher $h(n)$ means fewer nodes expanded ($h(n)=0$ degrades to UCS).

→ Examples: Manhattan distance, Euclidean distance, misplaced tiles (8-puzzle), straight-line distance (route finding).

Q24**What is the 8-Puzzle problem and how is it solved using A*?**

→ The 8-Puzzle has a 3×3 grid with 8 numbered tiles and one blank. The goal is to reach the goal configuration by sliding tiles. State space: $9!/2 = 181,440$ reachable states.

→ Heuristics for A*: h_1 = number of misplaced tiles (admissible). h_2 = sum of Manhattan distances of each tile from its goal position (admissible and more informed than h_1).

→ With h_2 , A* expands far fewer nodes and finds the optimal solution quickly.

Q25**★ IMPORTANT****What are Local Search Algorithms? Give examples.**

→ Local search algorithms operate by moving from current state to neighboring states without tracking paths. They are useful when the path itself is irrelevant (e.g., 8-queens) and when the state space is too large for systematic search.

→ Examples:

→ Hill Climbing: always moves to neighbor with highest value. Simple but gets stuck at local maxima.

→ Simulated Annealing: allows occasional bad moves (with decreasing probability) to escape local maxima.

→ Genetic Algorithms: population-based, uses crossover and mutation.

■ *These are also called optimization algorithms.*

Q26**★ IMPORTANT****What is Hill Climbing and what are its problems?**

→ Hill Climbing is a local search that always moves to the highest-valued neighboring state (steepest ascent) or the first better neighbor (simple hill climbing).

→ Problems: (1) Local Maxima: peak that is not the global maximum — algorithm gets stuck. (2) Ridges: sequence of local maxima that is difficult to navigate. (3) Plateaux: flat area with no uphill moves — random restart needed.

→ Solutions: Random Restarts (multiple starting points), Stochastic Hill Climbing, Simulated Annealing.

Q27**♦ PRACTICAL****What is Greedy Search and what are its applications as per the lab assignment?**

→ Greedy algorithms make locally optimal choices at each step hoping it leads to the global optimum. Applications in the lab:

→ Selection Sort: always select minimum element from unsorted part. $O(n^2)$.

→ Minimum Spanning Tree (Prim's/Kruskal's): greedily add minimum weight edge without forming cycle.

→ Dijkstra's Shortest Path: greedily pick minimum distance vertex.

→ Job Scheduling: sort by deadline/profit ratio and schedule.

■ *Greedy works optimally for MST and shortest path but not for all problems (e.g., 0/1 Knapsack needs DP).*

UNIT III · ADVERSARIAL SEARCH & GAMES

07 Hours

Course Outcomes: CO3, CO4

Q28**★ IMPORTANT****What is Adversarial Search? How does it differ from normal search?**

→ Adversarial search (game search) involves multiple agents with conflicting goals — each agent's gain is the other's loss (zero-sum). Unlike single-agent search where we look for a path to a goal, adversarial search must plan for an opponent who is trying to thwart you. The environment is competitive, and the agent must consider all possible opponent moves. Example games: Chess, Checkers, Go, Tic-tac-toe.

Q29**★ IMPORTANT****What is the Minimax Algorithm?**

→ Minimax is a recursive algorithm for optimal play in two-player zero-sum games. MAX player tries to maximize the utility; MIN player tries to minimize it.

→ Algorithm: At MAX nodes, pick action with maximum minimax value. At MIN nodes, pick action with minimum minimax value. At terminal nodes, return the utility.

→ Complexity: Time: $O(b^m)$, Space: $O(bm)$. For chess: $b \approx 35$, $m \approx 100$ — completely infeasible without pruning.

■ *Minimax assumes both players play optimally.*

Q30

★ IMPORTANT

✦ PRACTICAL

What is Alpha-Beta Pruning? How does it improve Minimax?

→ Alpha-Beta pruning eliminates branches in the minimax tree that cannot affect the final decision:

→ Alpha (α): best value MAX can guarantee so far (starts at $-\infty$)

→ Beta (β): best value MIN can guarantee so far (starts at $+\infty$)

→ Prune when: At MIN node, if current value $\leq \alpha$, stop (MAX won't choose this). At MAX node, if current value $\geq \beta$, stop (MIN won't choose this).

→ Complexity: Best case: $O(b^{(m/2)})$ — doubles the depth of search for the same time. Average case: $O(b^{(3m/4)})$. Worst case: $O(b^m)$ — same as minimax (no pruning).

■ *With perfect ordering of moves, Alpha-Beta can search twice as deep as pure minimax in the same time.*

Q31**What is Monte Carlo Tree Search (MCTS)?**

→ MCTS is a heuristic search algorithm that builds a search tree using random simulations (playouts). Four phases repeat iteratively:

→ (1) Selection: traverse tree using UCB1 formula to balance exploration/exploitation.

→ (2) Expansion: add one or more child nodes to the tree.

→ (3) Simulation: play out a random game from the new node to a terminal state.

→ (4) Backpropagation: update win/visit counts up the tree.

→ MCTS is used in Go (AlphaGo), requires no domain-specific heuristic, and works well in large state spaces.

Q32**What are Stochastic Games?**

→ Stochastic games introduce chance (randomness) into the game model — a third type of node, chance nodes (besides MAX and MIN), is added. The algorithm becomes Expectiminimax: at chance nodes, compute the expected value (weighted average of outcomes). Example: Backgammon — dice roll determines available moves. Complexity: $O(b^m \cdot a^m)$ where a = number of possible chance outcomes. The branching factor explodes, making deep search infeasible.

Q33

★ IMPORTANT

What are Constraint Satisfaction Problems (CSPs)?

→ A CSP is defined by: Variables $X = \{X_1, X_2, \dots, X_n\}$, Domains $D = \{D_1, D_2, \dots, D_n\}$ (possible values for each variable), Constraints C (restrictions on combinations of values).

→ Goal: Find an assignment of values to variables satisfying all constraints.

→ Examples: Map coloring, N-Queens, Sudoku, scheduling, timetabling.

■ *CSPs exploit the structure of the problem — most general search ignores internal structure of states.*

Q34

★ IMPORTANT

✦ PRACTICAL

How is Backtracking Search used for CSPs? Explain with N-Queens.

→ Backtracking search assigns values one variable at a time, backtracking when no valid value exists.

→ For N-Queens (N=8): Place one queen per row. For each row, try columns 1..N. Check: no two queens share same column or diagonal. If conflict, backtrack to previous row.

→ Algorithm:

```
function BACKTRACK(assignment, csp):
    if assignment complete, return assignment
    var = SELECT_UNASSIGNED_VARIABLE(csp)
    for each value in ORDER_DOMAIN_VALUES(var, assignment, csp):
        if value consistent with assignment:
            add var=value to assignment
            result = BACKTRACK(assignment, csp)
            if result ≠ failure, return result
    return failure
```

→ Complexity: $O(d^n)$ without heuristics, much better with arc consistency and MRV.

Q35

★ IMPORTANT

What is Constraint Propagation and Arc Consistency (AC-3)?

→ Constraint propagation infers domain reductions from constraints before or during search. Arc Consistency (AC-3) ensures that for every arc (X_i, X_j) , for each value of X_i , there exists at least one consistent value of X_j .

→ AC-3 Algorithm: Maintain a queue of arcs. For each arc, remove inconsistent values from X_i 's domain. If domain shrinks, re-add neighboring arcs. If any domain becomes empty, return failure.

→ Complexity of AC-3: $O(cd^3)$ where c = number of constraints, d = domain size.

■ AC-3 drastically reduces the search space before backtracking begins.

Q36**What heuristics improve CSP backtracking?**

→ Three key heuristics: (1) Minimum Remaining Values (MRV): choose the variable with the fewest legal values — the 'most constrained variable' heuristic, fails early. (2) Degree Heuristic (for ties): choose variable involved in the most constraints with remaining unassigned variables. (3) Least Constraining Value (LCV): for value ordering, choose value that rules out fewest choices for neighboring variables — keeps options open.

■ MRV chooses WHICH variable; LCV chooses WHICH value. Together they dramatically reduce search.

Q37**What are the limitations of game search algorithms?**

→ Limitations: (1) Exponential complexity — real games like Chess and Go have astronomically large state spaces. (2) Real-time constraints — agents must act within time limits. (3) Imperfect heuristic evaluation functions may mislead search. (4) Stochastic elements make exact analysis impossible. (5) Partially observable games (poker, bridge) require probabilistic reasoning. (6) Alpha-beta still exponential in worst case. Solutions: MCTS, endgame databases, iterative deepening with time limits, deep reinforcement learning (AlphaZero).

UNIT IV · KNOWLEDGE

07 Hours

Course Outcomes: CO3, CO4

Q38

★ IMPORTANT

What is a Knowledge-Based Agent?

→ A Knowledge-Based (KB) Agent maintains an explicit representation of the world (a knowledge base) and uses inference to derive new facts and decide actions. It has two key components:
 Knowledge Base (KB): a set of sentences in a formal language representing facts about the world.
 Inference Engine: derives new sentences from existing ones using logical rules.

→ TELL and ASK: new information is TELL'd to the KB; the agent ASKs the KB what action to take.

■ *KB agents can explain their reasoning — a major advantage over black-box ML models.*

Q39

★ IMPORTANT

What is the Wumpus World problem?

→ The Wumpus World is a classic AI environment used to illustrate KB agents: A 4×4 grid cave with a wumpus (monster), gold, and pits. Agent senses: Stench (adjacent to wumpus), Breeze (adjacent to pit), Glitter (gold nearby), Bump (hit wall), Scream (wumpus killed).

→ The agent must navigate safely using propositional logic to infer locations of pits and wumpus from percepts, without seeing them directly. It demonstrates how logical inference enables safe decision-making under uncertainty.

Q40

★ IMPORTANT

What is Propositional Logic? What are its components?

→ Propositional Logic (PL) is a formal language for representing facts using propositions (statements that are TRUE or FALSE). Components:

→ Atomic sentences: P, Q, R (basic propositions).

→ Connectives: $\neg$ (NOT), $\wedge$ (AND), $\vee$ (OR), $\Rightarrow$ (IMPLIES), $\Leftrightarrow$ (BICONDITIONAL).

→ Precedence: $\neg > \wedge > \vee > \Rightarrow > \Leftrightarrow$.

→ A model is an assignment of TRUE/FALSE to all propositional symbols. A sentence is valid (tautology) if true in all models, satisfiable if true in some model, unsatisfiable if true in no model.

Q41**★ IMPORTANT****What is a Truth Table and how is it used in propositional logic?**

→ A truth table enumerates all possible truth value assignments to propositional symbols and the resulting truth value of a compound sentence. For n symbols: 2^n rows.

→ Example for $P \rightarrow Q$: $P=T, Q=T \rightarrow T$; $P=T, Q=F \rightarrow F$; $P=F, Q=T \rightarrow T$; $P=F, Q=F \rightarrow T$.

→ Used for: Checking validity (all rows TRUE), satisfiability (any row TRUE), entailment ($KB \models \alpha$ means α is TRUE in all models where KB is TRUE).

■ Truth tables grow exponentially — practical for ≤ 20 symbols, beyond that use resolution or DPLL.

Q42**★ IMPORTANT****What is Propositional Theorem Proving?**

→ Theorem proving derives a sentence α from the KB without enumerating models. Key methods:

→ (1) Model Checking: enumerate all models and check if α is true whenever KB is true (TT-ENTAILS). Exponential but complete.

→ (2) Resolution: a complete inference procedure using the resolution rule — from $(A \vee B)$ and $(\neg B \vee C)$, derive $(A \vee C)$.

→ (3) DPLL Algorithm: efficient backtracking model checking with unit propagation and pure symbol heuristic — the basis of modern SAT solvers.

■ Resolution requires sentences in Conjunctive Normal Form (CNF).

Q43**★ IMPORTANT****What is First-Order Logic (FOL)? How does it differ from propositional logic?**

→ First-Order Logic (FOL) extends propositional logic with: Objects (constants: John, 2, Pune), Properties (predicates: Person(x), Prime(2)), Relations (predicates: Loves(John, Mary)), Functions (Father(John)), Quantifiers: $\forall$ (for all), $\exists$ (there exists).

→ Difference from PL: PL assumes a fixed set of true/false facts. FOL allows reasoning about objects, their properties, and relationships — far more expressive. FOL can represent: $\forall x \text{ Person}(x) \rightarrow \text{Mortal}(x)$ (All persons are mortal). PL cannot generalize over variables.

Q44**★ IMPORTANT****What are the Syntax and Semantics of First-Order Logic?**

→ Syntax: Terms: constants (John), variables (x), functions (Father(John)). Atomic sentences: Predicate(term1, term2, ...) e.g., Loves(John, Mary). Complex sentences: connected with $\wedge$, $\vee$, $\neg$, $\rightarrow$, $\leftrightarrow$. Quantified: $\forall x P(x)$, $\exists x P(x)$.

→ Semantics: Interpretation: maps constants to objects, predicates to relations, functions to functional relations. A sentence is TRUE or FALSE under an interpretation. Universal $\forall x P(x)$: $P(x)$ true for ALL objects in domain. Existential $\exists x P(x)$: $P(x)$ true for AT LEAST ONE object.

Q45**★ IMPORTANT****What is Knowledge Engineering in First-Order Logic?**

→ Knowledge Engineering is the process of building a KB for a domain using FOL:

→ (1) Identify the task — what questions must the KB answer?

→ (2) Assemble relevant knowledge — interview domain experts.

→ (3) Decide on vocabulary — choose predicates, functions, constants.

→ (4) Encode general knowledge — write axioms (rules).

→ (5) Encode the problem instance — assert specific facts.

→ (6) Query the KB — ask questions, examine answers.

→ (7) Debug — fix wrong answers by checking axioms.

■ *Example — encoding family relationships: $\text{Parent}(x,y)$, $\text{Sibling}(x,y)$, $\text{Ancestor}(x,y)$ with appropriate FOL axioms.*

Q46**What is Ontological Engineering?**

→ Ontological Engineering is the design of large-scale knowledge representations (ontologies) that capture a general model of the world. A general ontology defines categories (categories of objects), events (time-indexed occurrences), properties (object attributes), and relations (between objects and events).

→ Example: The Cyc project attempted to encode all of human commonsense knowledge. The Web Ontology Language (OWL) is used for semantic web ontologies.

■ *Ontologies are the upper-level knowledge shared across domains — domain-specific KBs extend the upper ontology.*

UNIT V · REASONING

07 Hours

Course Outcomes: CO4, CO5

Q47**★ IMPORTANT****What is Inference in First-Order Logic?**

→ Inference in FOL derives new sentences from the KB. Key methods:

→ (1) Universal Instantiation (UI): from $\forall x P(x)$, derive $P(c)$ for any constant c .

→ (2) Existential Instantiation (EI): from $\exists x P(x)$, introduce a new constant (Skolem constant).

→ (3) Propositionalization: convert FOL to propositional logic, then use PL inference — but leads to infinite propositions.

→ (4) Unification: find substitutions that make expressions identical.

→ (5) Generalized Modus Ponens (GMP): if KB contains $\forall x P(x) \rightarrow Q(x)$ and $P(\text{John})$, derive $Q(\text{John})$.

Q48**★ IMPORTANT****What is Unification in FOL?**

→ Unification finds a substitution θ (unifier) that makes two atomic sentences identical.

→ Example: $\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(\text{John}, \text{Jane})) = \{x/\text{Jane}\}$.

→ Most General Unifier (MGU): the substitution with fewest constraints — preferred for generality.

→ Unification Algorithm: Compare terms element by element. For variables, bind to corresponding term. Fails if: same variable is bound to two different terms (occurs check), or two different constants must unify.

■ *Unification is the backbone of Prolog and all FOL theorem provers.*

Q49**★ IMPORTANT****What is Forward Chaining in FOL?**

→ Forward Chaining starts from known facts and applies Modus Ponens forward to derive new facts until the query is proven or no new facts can be derived.

→ Algorithm: Start with KB facts. Find all rules whose premises are all known. Fire the rule — add conclusion to KB. Repeat until goal is found or KB stable.

→ Properties: Sound: all derived facts are true. Complete: will find any fact entailed by KB of Horn clauses. Data-driven: works well when many facts lead to few queries.

■ *Used in expert systems, production rule systems (CLIPS), and business rule engines.*

Q50**★ IMPORTANT****What is Backward Chaining in FOL?**

→ Backward Chaining starts from the query (goal) and works backward to see if the KB supports it, recursively proving subgoals.

→ Algorithm: To prove goal G , find rules with conclusion G . Try to prove all premises of each rule. If all premises provable, G is proven.

→ Properties: Sound and complete for Horn clause KBs. Goal-directed: only derives facts relevant to the query — more efficient than forward chaining when queries are specific.

■ *Prolog uses backward chaining with depth-first search and unification. Used in medical diagnosis systems.*

Q51**★ IMPORTANT****What is Resolution in FOL? What is its significance?**

→ Resolution is a complete inference procedure for FOL based on a single inference rule: from clauses $(A \vee B)$ and $(\neg B \vee C)$, derive $(A \vee C)$.

→ For FOL: convert all sentences to CNF (Conjunctive Normal Form), negate the query, add to KB, resolve clauses pairwise, if empty clause derived — original query is proven.

→ Significance: Resolution is refutation-complete — it can prove any entailed sentence by contradiction. It is the basis of logic programming (Prolog) and automated theorem provers.

■ *Resolution requires CNF — conversion involves pushing negations in, distributing OR over AND, and Skolemization.*

Q52

What is Knowledge Representation and what are its approaches?

→ Knowledge Representation (KR) is the field of encoding knowledge about the world in a form that AI systems can use for reasoning.

→ Approaches:

→ (1) Logical Representation: FOL, propositional logic — precise but computationally expensive.

→ (2) Semantic Networks: graph with nodes (objects) and labeled edges (relations).

→ (3) Frames: structured records with slots and fillers — precursor to objects.

→ (4) Production Rules: IF-THEN rules used in expert systems.

→ (5) Description Logic: basis of OWL for ontologies.

→ (6) Neural Representations: learned embeddings — not human-interpretable.

■ *Good KR balances expressiveness, inferential adequacy, and computational efficiency.*

Q53

What is Default Reasoning?

→ Default Reasoning handles conclusions that hold 'normally' but may have exceptions. Classical logic is monotonic — adding facts never removes conclusions. Default reasoning is non-monotonic — adding new information can invalidate previous conclusions.

→ Example: Default: Birds fly. Tweety is a bird. Conclude: Tweety flies. New fact: Tweety is a penguin. Retract: Tweety flies.

→ Formalisms: Default Logic (Reiter), Circumscription (McCarthy), Closed World Assumption (CWA) — what is not known to be true is assumed false. Used in databases and Prolog.

Q54

What are Categories, Objects, and Events in Ontological Engineering?

→ Categories (classes): natural groupings of objects sharing properties — e.g., Animal, Vehicle, Person. Subcategories create IS-A hierarchies (Dog IS-A Animal). Objects: individual instances belonging to categories — Fido IS-A Dog. Events: time-indexed occurrences — fluents are properties that change over time. The Situation Calculus represents how actions change the world: situation s , action $a \rightarrow \text{result}(a,s)$. Frame Problem: how to represent what does NOT change when an action occurs — a major challenge in commonsense reasoning.

UNIT VI · PLANNING & FUTURE OF AI

07 Hours

Course Outcomes: CO4, CO6

Q55**★ IMPORTANT**

What is Automated Planning?

→ Automated Planning is the AI task of finding a sequence of actions that transforms an initial state into a goal state. Unlike search, planning operates at a higher abstraction level with explicit action representations.

→ Classical Planning Assumptions: fully observable, deterministic, finite, static environment.

→ PDDL (Planning Domain Definition Language): standard language for specifying planning problems — defines types, predicates, actions (preconditions and effects), initial state, and goal.

■ *Planning is essential in robotics, logistics, game AI, and spacecraft operations (NASA uses planners).*

Q56**★ IMPORTANT**

What is Classical Planning and what is the STRIPS representation?

→ STRIPS (Stanford Research Institute Problem Solver) represents planning problems as:

→ State: set of positive ground atoms (facts true now).

→ Actions: preconditions (what must be true), effects (what becomes true/false).

→ Goal: set of atoms that must be true.

→ Example — Robot moving a block:

→ ACTION: MoveBlock(b, from, to)

→ PRECOND: $\text{On}(b, \text{from}) \wedge \text{Clear}(\text{to}) \wedge \text{Clear}(b)$

→ EFFECT: $\text{On}(b, \text{to}) \wedge \neg \text{On}(b, \text{from}) \wedge \neg \text{Clear}(\text{to}) \wedge \text{Clear}(\text{from})$

→ STRIPS solves planning problems by forward/backward search in the state space.

Q57

What are Heuristics for Planning?

→ Planning heuristics estimate cost to reach the goal from current state:

→ (1) Ignore Preconditions: drop all preconditions — lower bound on steps needed.

→ (2) Ignore Delete Effects: only consider positive effects — relaxed problem is easier.

→ (3) Level-Sum (Graphplan): sum of levels at which each goal first appears in planning graph.

→ (4) Pattern Databases: precompute exact costs for subproblems.

■ *The delete-relaxation heuristic (h^+) is the most commonly used — it's admissible and computable in polynomial time.*

Q58

What is Hierarchical Planning (HTN)?

→ Hierarchical Task Network (HTN) planning decomposes high-level tasks into subtasks using methods until primitive actions are reached.

→ Example: Task: make_breakfast → Methods: {make_eggs_and_toast, make_cereal_and_milk}.
make_eggs_and_toast → {boil_eggs, make_toast}.

→ Advantages: More efficient than flat planning (uses domain knowledge), solutions are human-interpretable, can handle very complex plans. Used in: military planning, robot task execution, game AI (NPCs in video games).

■ HTN planning is used in SIRI and virtual assistants for task decomposition.

Q59

What is Planning in Nondeterministic Domains?

→ Nondeterministic planning handles uncertainty — actions may have multiple possible outcomes.

→ Approaches:

→ (1) Contingency Planning: plan with branches for each possible outcome (and-or trees). Generates a conditional plan.

→ (2) Online Replanning: execute plan, observe outcome, replan if needed.

→ (3) Conformant Planning: find a plan that works regardless of uncertainty — most conservative.

→ (4) MDP (Markov Decision Process): probabilistic model, find policy (mapping state→action) maximizing expected reward.

■ Nondeterministic planning is crucial for robotics where sensors and actuators are imperfect.

Q60

★ IMPORTANT

What are the Limits and Ethics of AI?

→ Limits: AI cannot truly understand meaning (Chinese Room argument by Searle). Brittleness — fails on out-of-distribution inputs. Requires massive data and computation. Cannot reliably reason causally. No genuine consciousness or self-awareness.

→ Ethical concerns:

→ (1) Bias: training data biases encoded in models (facial recognition errors on minorities).

→ (2) Privacy: mass surveillance enabled by AI.

→ (3) Accountability: who is responsible when AI fails?

→ (4) Autonomy: autonomous weapons systems (lethal AI).

→ (5) Economic disruption: job displacement.

→ (6) Existential risk: misaligned superintelligence (long-term concern).

■ AI ethics is now a formal field — Anthropic, DeepMind, and OpenAI all have safety teams.

Q61

★ IMPORTANT

What is the Future of AI and its key components?

→ Near-term (2025–2030): Multimodal AI (text+image+audio+video), autonomous agents, personalized medicine AI, AI-powered scientific discovery.

→ Long-term: Artificial General Intelligence (AGI) — AI that can perform any intellectual task a human can. AI Architectures: Neural-symbolic systems (combining deep learning with symbolic reasoning), neuromorphic computing, quantum AI.

→ AI Components evolving: Better reasoning (chain-of-thought), planning, memory (long-context models), tool use (function calling), and learning from human feedback (RLHF).

■ *The SPPU syllabus specifically includes Samsung using Big Data and AI as the Unit VI case study.*

PART I — LAB PRACTICAL QUESTIONS

Group A (Compulsory) · Group B · Group C · Code · Complexity · Output · Applications

AI Lab Assignments — Group A, B & C

Q1

★ IMPORTANT

↗ PRACTICAL

Explain your DFS implementation. What are the function parameters and what does each do?

→ Your DFS function signature: `dfs(graph, visited, node)`

→ graph: adjacency list stored as a Python dict — e.g., `{0: [1, 2], 1: [0, 3]}`

→ visited: a Python `set()` passed by reference — tracks already-visited nodes to avoid infinite loops

→ node: the current node being processed

```
def dfs(graph, visited, node):  
    if node not in visited:  
        print(node, end=' ')  
        visited.add(node)  
        for neighbor in graph[node]:  
            if neighbor not in visited:  
                dfs(graph, visited, neighbor)
```

→ Key detail: the visited check is done BEFORE printing — so the node is printed only once.

→ Why pass visited as a parameter? So the same set is shared across all recursive calls. If a new `set()` was created each call, already-visited nodes would be revisited causing infinite recursion.

■ Called as `dfs(graph, set(), start)` — a fresh empty set is passed at first call. Your inner `if neighbor not in visited` is redundant (the outer check handles it) but harmless.

Q2

★ IMPORTANT

✦ PRACTICAL

Trace the DFS output for a sample graph. What traversal order does DFS produce?

→ Example: graph = {0:[1,2], 1:[0,3], 2:[0], 3:[1]}, start=0

→ DFS Trace step by step:

Call dfs(graph, {}, 0): 0 not in visited → print 0, add 0

neighbor 1: call dfs(graph, {0}, 1)

1 not visited → print 1, add 1

neighbor 0: already visited, skip

neighbor 3: call dfs(graph, {0,1}, 3)

print 3, add 3. neighbor 1 visited. return.

neighbor 2: call dfs(graph, {0,1,3}, 2)

print 2, add 2. neighbor 0 visited. return.

→ Output: 0 1 3 2

→ Key property: DFS goes DEEP before WIDE — fully explores one branch before moving to the next sibling.

→ Time Complexity: $O(V+E)$. Space Complexity: $O(V)$ for visited set + $O(V)$ implicit recursion call stack.**Q3**

★ IMPORTANT

✦ PRACTICAL

Explain your BFS implementation. How does it differ from your DFS structurally?

→ Your BFS function: bfs(graph, start)

```
def bfs(graph, start):
```

```
    visited = set()
```

```
    queue = [start] # plain Python list used as FIFO queue
```

```
    visited.add(start) # mark BEFORE entering queue (not after popping)
```

```
    while queue:
```

```
        node = queue.pop(0) # dequeue from FRONT —  $O(n)$  operation
```

```
        print(node, end=' ')
```

```
        for neighbor in graph[node]:
```

```
            if neighbor not in visited:
```

```
                visited.add(neighbor) # mark when enqueued
```

```
                queue.append(neighbor) # enqueue at BACK
```

→ Critical difference from DFS:

→ 1. BFS marks visited WHEN ENQUEUED — prevents same node appearing twice in queue.

→ 2. DFS marks visited WHEN FIRST ENTERED recursively.

→ 3. BFS uses queue (FIFO); DFS uses call stack (LIFO).

■ *queue.pop(0) is $O(n)$ — use collections.deque with popleft() for $O(1)$ in production.*

Q4

★ IMPORTANT

✦ PRACTICAL

Trace the BFS output for the same graph. What level-order means?

→ Example: graph = {0:[1,2], 1:[0,3], 2:[0], 3:[1]}, start=0

→ BFS Trace:

Start: queue=[0], visited={0}

Pop 0, print 0. Neighbors 1,2 not visited → add both.

queue=[1,2], visited={0,1,2}

Pop 1, print 1. Neighbor 0 visited. Neighbor 3 → add.

queue=[2,3], visited={0,1,2,3}

Pop 2, print 2. Neighbor 0 visited. queue=[3]

Pop 3, print 3. Neighbor 1 visited. queue=[]

→ Output: 0 1 2 3

→ Level-order: BFS visits all nodes at distance 1 from start (nodes 1,2) before distance 2 (node 3).

→ BFS guarantees shortest path (fewest hops) in unweighted graphs — DFS does NOT guarantee this.

→ Time: $O(V+E)$. Space: $O(V)$ for visited + $O(V)$ for queue (at most V nodes in queue).

Q5

✦ PRACTICAL

How does your code take graph input from the user? What data structure is built?

→ Input section from your code:

```
n = int(input('Enter number of vertices: '))
```

```
graph = {}
```

```
for i in range(n):
```

```
    neighbors = list(map(int, input(f'Enter the neighbors of node {i}:- ').split()))
```

```
    graph[i] = neighbors
```

→ Builds a Python dictionary (adjacency list): keys = node integers 0 to $n-1$, values = list of neighbor integers.

→ Example for 4-node undirected graph: enter 1 2 for node 0, enter 0 3 for node 1, etc.

→ Result: graph = {0:[1,2], 1:[0,3], 2:[0], 3:[1]}

→ Why adjacency list over matrix? Space: $O(V+E)$ vs $O(V^2)$. Faster neighbor iteration for sparse graphs.

■ For undirected graphs, the user must enter edges symmetrically — if 0 connects to 1, node 0's list must contain 1 AND node 1's list must contain 0.

Q6

★ IMPORTANT

✦ PRACTICAL

Explain your Selection Sort. Why is Selection Sort a Greedy algorithm?

→ Your Selection Sort code:

```
for i in range(n):  
    min_index = i  
    for j in range(i + 1, n):  
        if arr[j] < arr[min_index]:  
            min_index = j  
    arr[i], arr[min_index] = arr[min_index], arr[i] # swap
```

→ Greedy reasoning: at each step i , the locally optimal (greedy) choice is to pick the minimum of $arr[i..n-1]$ and commit it to position i permanently — no backtracking ever.

→ Complexity: Time $O(n^2)$ — n iterations of outer loop, up to $n-1$ comparisons each inner loop.

→ Space: $O(1)$ — in-place, only `min_index` variable as extra space.

→ Key property: exactly $n-1$ swaps total regardless of input — better than Bubble Sort which can do $O(n^2)$ swaps.

→ Output: Sorted array printed as a Python list.

Q7

★ IMPORTANT

✦ PRACTICAL

Trace your Selection Sort for input [64, 25, 12, 22, 11] step by step.

→ Input: `arr = [64, 25, 12, 22, 11]`, $n=5$

`i=0`: scan `arr[1..4]`, min at index 4 (value 11). Swap `arr[0], arr[4]`. →
`[11, 25, 12, 22, 64]`

`i=1`: scan `arr[2..4]`, min at index 2 (value 12). Swap `arr[1], arr[2]`. →
`[11, 12, 25, 22, 64]`

`i=2`: scan `arr[3..4]`, min at index 3 (value 22). Swap `arr[2], arr[3]`. →
`[11, 12, 22, 25, 64]`

`i=3`: scan `arr[4..4]`, min at index 3 (value 25). No swap (`min_index==i`). →
`[11, 12, 22, 25, 64]`

`i=4`: inner loop doesn't run. Done.

→ Output: Sorted array: `[11, 12, 22, 25, 64]`

■ Selection Sort is NOT stable — equal elements may swap relative order. It is in-place ($O(1)$ extra space). Always $O(n^2)$ regardless of whether input is already sorted.

Q8

★ IMPORTANT

✦ PRACTICAL

Explain your A* implementation. What do open_list, g, and parent store?

→ Your A* data structures:

→ open_list: plain Python list of node names not yet finalized — nodes discovered but not yet expanded.

→ g dict: maps each node to its best known actual cost from start. Initialized $g[start]=0$.

→ parent dict: maps each node to the node it was reached from — used to reconstruct the path. $parent[start]=None$.

```
def a_star(graph, h, start, goal):
```

```
    open_list = [start]
```

```
    g = {}; g[start] = 0
```

```
    parent = {}; parent[start] = None
```

```
    while open_list:
```

```
        current = open_list[0]
```

```
        for node in open_list: # linear scan for min f(n)
```

```
            if g[node]+h[node] < g[current]+h[current]:
```

```
                current = node
```

→ $f(n) = g(n) + h(n)$ is computed inline as $g[node]+h[node]$. The node with smallest f is selected each iteration.

■ Your open_list is a list — selecting minimum is $O(|open_list|)$ per step. A heapq priority queue would make this $O(\log n)$.

Q9

★ IMPORTANT

✦ PRACTICAL

What is f(n), g(n), h(n) in your A* code? How are they calculated?

→ $g(n)$: actual cost from start to node n — stored in the g dict.

```
new_cost = g[current] + cost
```

```
if neighbor not in g or new_cost < g[neighbor]:
```

```
    g[neighbor] = new_cost
```

→ Updated when a cheaper path to neighbor is found — relaxation step.

→ $h(n)$: heuristic estimate from n to goal — stored in h dict, provided by user as integer input.

→ $f(n) = g(n) + h(n)$: total estimated cost through n — computed inline: $g[node]+h[node]$.

→ Admissibility: $h(n)$ must never overestimate the true cost. If the user enters admissible h values, your A* is guaranteed to return the optimal path.

→ If $h[node]=0$ for all nodes: A* degenerates to Uniform Cost Search (Dijkstra's) — explores all directions equally.

■ Your code does NOT have a separate closed/visited set — a node can be re-added to open_list if a cheaper path is found (the $g[neighbor]$ check handles this).

Q10

★ IMPORTANT

♦ PRACTICAL

How does path reconstruction work in your A* code? Trace through it.

→ After reaching goal, path is reconstructed by backtracking through parent dict:

```
path = []  
while current is not None:  
    path.append(current) # add current node to path  
    current = parent[current] # move to parent  
path.reverse() # reverse: goal-to-start becomes start-to-goal  
print('Shortest Path:', path)
```

→ Why reverse? The while loop builds the path BACKWARDS (goal → ... → start). Reversing gives the correct direction.

→ Example: parent = {'A':None, 'B':'A', 'D':'B'}. Goal='D'

```
current=D → path=[D], current=B  
current=B → path=[D,B], current=A  
current=A → path=[D,B,A], current=None (stop)  
path.reverse() → ['A', 'B', 'D']
```

→ Output: Shortest Path: ['A', 'B', 'D']

→ If goal never reached: open_list empties → prints 'No Path Found'.

Q11

♦ PRACTICAL

What is the difference between your A* and Dijkstra's? When would you use each?

→ Dijkstra's: expands by $g(n)$ only — no heuristic. Explores all directions equally — like a growing circle. Guaranteed optimal for non-negative edge weights. Use when: no domain knowledge for a heuristic exists.

→ Your A*: expands by $f(n)=g(n)+h(n)$. Heuristic guides search toward goal — explores fewer nodes in practice. Guaranteed optimal if h is admissible. Use when: a good heuristic is available (e.g., straight-line distance for maps, Manhattan distance for grid).

→ Equivalence: if all $h[\text{node}]=0$ entered by user, your A* behaves exactly like Dijkstra's.

→ Example: A* with Manhattan distance finds path in 8-puzzle exploring ~500 nodes. Dijkstra's explores all ~181,000 reachable states.

■ Both are $O((V+E) \log V)$ with a priority queue, but A* expands far fewer nodes with a good heuristic.

Q12

★ IMPORTANT

✦ PRACTICAL

Explain your N-Queens is_safe() function. What are the three safety checks?

→ Your is_safe(board, row, column, n) performs three checks:

→ Check 1 — Same column:

```
for i in range(row):
    if board[i] == column: return False
```

→ board[i] stores the column of the queen in row i. If any earlier row has a queen in the same column → conflict.

→ Check 2 — Upper-left diagonal:

```
for i, j in zip(range(row, -1, -1), range(column, -1, -1)):
    if board[i] == j: return False
```

→ Walks diagonally upward-left from (row, column). If board[i]==j means queen is on this diagonal.

→ Check 3 — Upper-right diagonal:

```
for i, j in zip(range(row, -1, -1), range(column, n)):
    if board[i] == j: return False
```

→ Walks diagonally upward-right. Only rows ABOVE are checked — no queen placed below current row yet.

→ Returns True only if all three checks pass.

■ A simpler equivalent: check board[i]==column (same column) OR abs(board[i]-column)==abs(i-row) (diagonal). Your zip approach is more explicit but equivalent.

Q13

★ IMPORTANT

✦ PRACTICAL

Explain your nqueens() backtracking logic and what the board list represents.

→ board: list of size n, board[row]=column means queen in that row is at that column. Initialized to [-1]*n (no queen placed).

```
def nqueens(board, row, n):
    if row == n: # base case: all n queens placed
        print(board) # print this solution
        return
    for column in range(n):
        if is_safe(board, row, column, n):
            board[row] = column # PLACE queen
            nqueens(board, row + 1, n) # recurse to next row
            board[row] = -1 # BACKTRACK – undo placement
```

→ Base case: row==n means all rows 0..n-1 are filled — a complete solution found, print it.

→ Backtracking: after recursion returns, board[row]=-1 resets that row and the loop tries the next column.

→ Your code prints ALL solutions, not just the first. Called as: nqueens(board, 0, n).

→ Output: n=4 → 2 solutions: [1,3,0,2] and [2,0,3,1]. n=8 → 92 solutions.

■ [1,3,0,2] means row0-col1, row1-col3, row2-col0, row3-col2.

Q14

♦ PRACTICAL

What is the time complexity of your N-Queens? Why is backtracking efficient here?

→ Naive (no backtracking): try all n^n placements → $O(n^n)$ — completely infeasible for $n=20$.

→ With backtracking: prune branches early via `is_safe()`. Effective complexity closer to $O(n!)$ but much better in practice.

→ Actual solution counts: $n=4$: 2 solutions. $n=6$: 4. $n=8$: 92. $n=12$: 14,200. $n=16$: 14,772,512.

→ Your `is_safe()` runs $O(n)$ per call (linear scan of rows above). Total: $O(n * n!)$ worst case, but heavy pruning makes it practically $O(n!)$ or better.

→ Space Complexity: $O(n)$ for board list + $O(n)$ recursion stack depth — very memory efficient.

→ Why backtracking works well here: constraint propagation — placing a queen eliminates many subsequent column choices. On average, only ~1.8 columns per row are valid as n grows.

■ *N-Queens is a classic CSP (Constraint Satisfaction Problem). Your backtracking IS the standard CSP solution approach.*

Q15

★ IMPORTANT

♦ PRACTICAL

Explain your Graph Coloring `is_safe()` and `solve()` functions with the adjacency matrix.

→ Your graph coloring uses an ADJACENCY MATRIX: `graph[node][i]==1` means edge between node i and node.

```
def is_safe(node, color, graph, colors, c):
    for i in range(len(graph)):
        if graph[node][i] == 1 and colors[i] == c:
            return False
    return True
```

→ Checks: for every vertex i adjacent to node (`graph[node][i]==1`), if i is already colored with c → unsafe.

```
def solve(graph, m, colors, node):
    if node == len(graph): # all nodes colored
        print('Solution Found:', colors); return True
    for c in range(1, m + 1): # try colors 1..m
        if is_safe(node, color, graph, colors, c): # BUG: 'color' undefined
            colors[node] = c
            if solve(graph, m, colors, node + 1): return True
            colors[node] = 0 # backtrack
    return False
```

■ *Bug in your code — `is_safe` called with undefined variable 'color' as 2nd arg. Correct call: `is_safe(node, c, graph, colors, c)`. The 'color' parameter in `is_safe()` signature is unused — the 'c' parameter does the actual work.*

Q16

★ IMPORTANT

✦ PRACTICAL

What input does your Graph Coloring expect? Give a sample run with output.

→ Input format: number of vertices n , then n rows of n space-separated integers (0 or 1) forming adjacency matrix, then m (number of colors).

→ Example — 4-node cycle graph:

$n = 4$

Row 0: 0 1 0 1

Row 1: 1 0 1 0

Row 2: 0 1 0 1

Row 3: 1 0 1 0

$m = 2$

→ This cycle can be 2-colored (bipartite): alternating color 1 and 2.

→ Expected Output: Solution Found: [1, 2, 1, 2]

→ Meaning: node0=color1, node1=color2, node2=color1, node3=color2.

→ If $m=1$ (one color) for the same graph → No Solution (adjacent nodes would need same color).

→ colors array: initialized $[0]*n$ where 0 means uncolored.

→ Time Complexity: $O(m^n)$ worst case. Space: $O(n)$ colors array + $O(n)$ recursion depth.

■ Chromatic number (minimum m for a solution) is NP-hard to compute. Your code checks sufficiency for a given m .

Q17

★ IMPORTANT

✦ PRACTICAL

Explain your Chatbot. What library is used, what is Chat, and what is reflections?

→ Your chatbot uses NLTK (Natural Language Toolkit):

```
import nltk
```

```
from nltk.chat.util import Chat, reflections
```

```
chatbot = Chat(pair, reflections)
```

```
chatbot.converse()
```

→ Chat class: takes a list of [regex_pattern, response_list] pairs. When user types input, patterns are tested in ORDER using `re.search()`. First match wins and a response is returned.

→ reflections: built-in dict for pronoun swapping — maps 'i am' → 'you are', 'my' → 'your', etc. Makes responses grammatically natural.

→ Domain: Banking customer service chatbot.

→ Patterns in your code:

```
r'my name is (.*)' → 'Hello %1, how may i help you?' [capture group]
```

```
r'hello|hi|howdy' → 'Hello there, welcome to the bank'
```

```
r'open an account' → 'visit the branch'
```

```
r'send money' → 'use our app to send money'
```

```
r'bye' → 'bye'
```

■ %1 in response is replaced by the first captured group from the regex — e.g., user: 'my name is Raj'

→ response: 'Hello Raj, how may i help you?'

Q18

★ IMPORTANT

✦ PRACTICAL

What does chatbot.converse() do? What happens if no pattern matches?

→ chatbot.converse() runs an infinite loop:

```
def converse(self, quit='quit'):
    input_str = input('> ')
    while input_str != quit:
        print(self.respond(input_str))
        input_str = input('> ')
```

→ It reads user input, calls respond() which tries each pattern in order, prints the matched response, and loops until user types 'quit'.

→ If no pattern matches: Chat.respond() returns None. converse() then prints None — the user sees 'None' on screen. This is a limitation of your current chatbot — no fallback pattern.

→ Fix: add as the LAST pair:

```
[ r'(.*)' , ['I did not understand. Please rephrase.',
'Could you please clarify your query?'] ]
```

→ r'(.*)' matches ANY string — must be last so specific patterns are tried first.

■ If multiple responses are given in the list, Chat randomly picks one — reducing repetitiveness.

Q19

✦ PRACTICAL

What are the limitations of your rule-based chatbot vs an AI-powered chatbot?

→ Limitations of your NLTK rule-based chatbot:

→ 1. Pattern coverage: only handles 5 specific inputs. Any other input gets None or no response.

→ 2. No context memory: each input matched independently — no conversation history maintained. Cannot handle multi-turn dialogue.

→ 3. Exact regex match: user must type close to the pattern — typos or synonyms ('transfer money' won't match 'send money').

→ 4. No learning: static patterns — cannot improve from user interactions.

→ 5. No intent understanding: just pattern matching, not semantic understanding.

→ AI-powered chatbot improvements:

→ Intent classification: ML model (BERT, spaCy) classifies intent regardless of exact wording.

→ Entity extraction: NER identifies account numbers, amounts, dates from free text.

→ Dialogue management: Rasa, Dialogflow maintain conversation state.

■ For a banking chatbot handling sensitive data, AI models also need strict security — your rule-based approach is actually safer (no ML hallucinations, fully deterministic).

Q20

★ IMPORTANT

✦ PRACTICAL

Explain your Expert System. What domain, what are QUESTIONS and THRESHOLD?

→ Domain: Stock Market Trading advisor — helps decide BUY, HOLD, or SELL.

QUESTIONS = [

'Is the stock price increasing steadily?',

'Is the trading volume high?',

'Is market sentiment positive?',

'Is the company showing good financial results?',

'Is the overall market bullish?',

'Is news about the stock positive?'

]

THRESHOLD = {'Buy': 30, 'Hold': 15}

→ 6 questions covering key stock analysis indicators: price trend, volume, market sentiment, financial health, overall market, news.

→ Max possible score = 6 questions × 10 = 60 points.

→ Decision rules: score ≥ 30 → BUY. score ≥ 15 → HOLD. score < 15 → SELL.

■ This is forward chaining — all evidence (user answers) collected first, then one conclusion fired. This maps to the Stock Market Trading Expert System option in SPPU Group C syllabus.

Q21

★ IMPORTANT

✦ PRACTICAL

Walk through the scoring logic in `expert_system()`. What does each block do?

→ Full scoring flow:

`score = 0``for i in questions: # iterate each question``print(i + 'y/n:-')``ans = input('')``if ans == 'y':``print('give a range between 1 and 10:- ')``ans2 = input('')``while((not ans2.isnumeric()) or int(ans2)<1 or int(ans2)>10):``print('invalid input, try again')``ans = input('') # BUG: should be ans2 = input('')``score += int(ans2)`

→ For each question: if user answers 'y', they rate their confidence 1-10. That rating adds to score. If 'n', skip (0 added).

→ Validation: while loop checks 3 conditions — not numeric, < 1, > 10 — using OR. All must be False to exit loop.

→ Bug: inside while loop, `ans = input()` re-reads into 'ans' not 'ans2'. Fix: change `ans` to `ans2` so the loop re-evaluates the new rating.

→ Final decision: threshold comparisons in order Buy → Hold → else Sell.

■ *Mention this bug confidently in viva — examiners appreciate students who can identify and explain bugs in their own code.*

Q22

★ IMPORTANT

♦ PRACTICAL

What is the output of your Expert System? Give a sample bullish and bearish run.

→ Bullish scenario (all relevant indicators positive):

Is the stock price increasing steadily? y → rating 9 (score=9)

Is the trading volume high? y → rating 8 (score=17)

Is market sentiment positive? y → rating 7 (score=24)

Is the company showing good financials? y → rating 8 (score=32)

Remaining 2 questions answered n → score stays 32

→ Score=32 ≥ THRESHOLD['Buy']=30:

Strong BUY signal

You can consider buying this stock.

→ Bearish scenario (all 'n'):

→ Score=0 < 15:

SELL signal

Risk is high, consider selling or avoiding this stock.

→ Hold scenario (score=20):

HOLD signal

Better to wait and watch the market.

→ Edge cases: score exactly 30 → BUY. Score exactly 15 → HOLD. Max score 60 (all y, all 10).
Min score 0 (all n).

Q23

♦ PRACTICAL

Map the components of a classical Expert System to your code.

→ Classical Expert System has 4 components:

→ (1) Knowledge Base: QUESTIONS list (domain facts — what indicators matter for stocks) and THRESHOLD dict (domain rules — what score means what action). This IS the encoded expertise of a stock analyst.

→ (2) Inference Engine: the for loop + if/elif/else decision block in expert_system() — applies forward chaining, accumulates evidence (score), fires conclusion rule once.

→ (3) User Interface: input() prompts and print() statements — the Q&A; session where the expert system elicits facts from the user.

→ (4) Explanation Module: the print statements after decision — 'Strong BUY signal, You can consider buying this stock' explain the recommendation (though a richer explanation would say which factors drove the decision).

→ Forward vs Backward chaining: your system uses FORWARD chaining — gather all evidence then conclude. Backward chaining would start from hypothesis (Is this a BUY?) and ask only the minimum questions needed to confirm or deny it.

■ Your scoring approach (weighted evidence) is more nuanced than pure IF-THEN rules — it models uncertainty better.

PART II — CLOUD COMPUTING

310254(C) · All 6 Theory Units + Lab Assignments

UNIT I · INTRODUCTION TO CLOUD COMPUTING

07 Hours

Course Outcomes: CO1

Q1

★ IMPORTANT

What is Cloud Computing? What are its essential characteristics?

→ Cloud Computing is the on-demand delivery of computing resources (servers, storage, databases, software) over the internet with pay-as-you-go pricing. NIST defines five essential characteristics:

→ (1) On-demand self-service: provision resources without human interaction.

→ (2) Broad network access: available over the network via standard mechanisms.

→ (3) Resource pooling: multi-tenant model, resources dynamically assigned.

→ (4) Rapid elasticity: scale up/down quickly.

→ (5) Measured service: resource usage monitored and billed.

■ *The IBM Cloud Computing Model is the Unit I case study.*

Q2

★ IMPORTANT

What are the Pros and Cons of Cloud Computing?

→ Pros: Cost savings (no upfront capital expenditure), scalability (auto-scale), reliability (99.99% SLA), accessibility (anywhere, any device), automatic updates, disaster recovery, collaboration, green computing (shared resources).

→ Cons: Security and privacy concerns (data outside your premises), vendor lock-in, downtime (outages affect all), limited control and customization, internet dependency, compliance challenges (data sovereignty), hidden costs (egress fees).

■ *Exam tip — mention both sides with examples (AWS S3 outage in 2017 took down many websites).*

Q3

★ IMPORTANT

What are the Cloud Service Models — SaaS, PaaS, IaaS?

→ IaaS (Infrastructure as a Service): provides virtualized computing resources — VMs, storage, networking. User manages OS and above. Example: AWS EC2, Azure VMs, Google Compute Engine.

→ PaaS (Platform as a Service): provides platform for developing, running, deploying applications. User manages application and data. Example: Google App Engine, Heroku, Azure App Service.

→ SaaS (Software as a Service): fully managed software accessed via browser. User just uses the app. Example: Gmail, Salesforce, Microsoft 365, Zoom.

■ *Remember with the pizza analogy — IaaS: kitchen rented; PaaS: ingredients provided; SaaS: pizza delivered.*

Q4**★ IMPORTANT****What are the Cloud Deployment Models?**

- Public Cloud: owned by cloud provider, shared among multiple tenants. Most cost-effective. Example: AWS, Azure, GCP.
- Private Cloud: owned and operated by a single organization. More control and security. Example: OpenStack in an enterprise data center.
- Hybrid Cloud: combination of public and private — sensitive data on private, scalable workloads on public. Example: Azure Arc, AWS Outposts.
- Community Cloud: shared by multiple organizations with similar requirements (government clouds, healthcare clouds).
- Multi-Cloud: using services from multiple cloud providers to avoid lock-in.

Q5**What is the Seven-Step Migration Model for moving to the Cloud?**

- Step 1: Assess — evaluate current applications, data, and infrastructure.
 - Step 2: Isolate — identify dependencies between components.
 - Step 3: Map — map existing components to cloud services.
 - Step 4: Re-architect — redesign components as needed for cloud.
 - Step 5: Augment — add cloud-native features (auto-scaling, load balancing).
 - Step 6: Test — thoroughly test migrated system.
 - Step 7: Optimize — continuously monitor and optimize cost and performance.
- *The 5 R's of migration: Rehost (lift and shift), Replatform, Refactor, Repurchase, Retire.*

Q6**What is the Cloud Computing Logical Architecture?**

- The logical architecture has four layers:
- (1) Physical Layer: actual hardware — servers, storage, networking in data centers.
- (2) Infrastructure Layer (IaaS): virtualized compute, storage, networking resources.
- (3) Platform Layer (PaaS): middleware, runtime environments, databases, development tools.
- (4) Application Layer (SaaS): end-user applications.
- The Reference Model (NIST) adds a cross-cutting Security layer, a Management layer (orchestration), and Service Brokers (intermediaries between consumers and providers).

UNIT II · DATA STORAGE AND CLOUD COMPUTING

07 Hours

Course Outcomes: CO2

Q7**★ IMPORTANT**

What are the types of Enterprise Data Storage?

- DAS (Direct Attached Storage): storage directly connected to one server via SCSI/SAS. Simple but not shareable.
- SAN (Storage Area Network): high-speed network (Fibre Channel or iSCSI) dedicated to storage — block-level access. Expensive but high performance.
- NAS (Network Attached Storage): file-level access over Ethernet using NFS or SMB protocols. Easy to share across multiple servers.
- Cloud Storage: object, block, or file storage provided as a service over the internet — examples: AWS S3 (object), EBS (block), EFS (file).

Q8**★ IMPORTANT**

What is Cloud Storage? What are its types?

- Cloud Storage stores data on remote servers managed by a cloud provider, accessible via API or internet.
- Types:
 - (1) Object Storage: stores data as objects with metadata and unique ID. Best for unstructured data. Example: AWS S3, GCS — massively scalable, no hierarchy.
 - (2) Block Storage: stores data in fixed-size blocks, like a virtual hard drive. Example: AWS EBS — used for databases, VMs.
 - (3) File Storage: hierarchical filesystem accessible via NFS/SMB. Example: AWS EFS — shared file system for multiple instances.
- *S3 is the most widely used cloud storage — unlimited scale, 11-nines durability.*

Q9

What is HDFS (Hadoop Distributed File System)?

- HDFS is a distributed file system designed for storing very large files across commodity hardware. Key design:
- Blocks: files split into 128MB blocks (default) distributed across nodes.
- NameNode: master node storing metadata (file names, block locations).
- DataNode: slave nodes storing actual data blocks.
- Replication: each block replicated 3 times (default) for fault tolerance.
- Write once, read many: optimized for sequential reads (batch processing).
- *The lab mini-project assignment requires implementing basic HDFS operations (divide file into blocks, upload/download encrypted) using open-source technologies.*

Q10**What is Data-Intensive Cloud Computing?**

→ Data-intensive computing involves processing and analyzing massive datasets that exceed single-machine capabilities. Technologies:

→ MapReduce: programming model for processing large datasets in parallel — Map phase (split and process), Reduce phase (aggregate results).

→ Apache Spark: in-memory distributed computing, 100x faster than MapReduce.

→ HBase: NoSQL database on top of HDFS for random, real-time access.

→ Cassandra: distributed wide-column store for high-availability, no single point of failure.

■ *Case studies in syllabus: Online Book Marketing Service (distributed catalog search), Online Photo Editing (distributed image processing).*

Q11**What is Distributed Data Storage?**

→ Distributed Data Storage spreads data across multiple nodes for scalability, availability, and fault tolerance. Key concepts:

→ Sharding (Horizontal Partitioning): split data across nodes by key range or hash.

→ Replication: maintain copies on multiple nodes — synchronous (strong consistency) or asynchronous (eventual consistency).

→ CAP Theorem: a distributed system can guarantee at most 2 of: Consistency, Availability, Partition Tolerance.

→ Consistency Models: Strong (all nodes see same data), Eventual (nodes eventually converge), Causal (causally related operations in order).

UNIT III · VIRTUALIZATION IN CLOUD COMPUTING

07 Hours

Course Outcomes: CO3

Q12

★ IMPORTANT

What is Virtualization? What are its types?

→ Virtualization creates virtual (software-based) versions of physical resources, allowing multiple virtual instances on one physical machine.

→ Types:

→ (1) Full Virtualization (VMware): complete simulation of hardware — guest OS runs unmodified. Hypervisor translates all privileged instructions.

→ (2) Paravirtualization (Xen): guest OS modified to use hypervisor API (hypercalls). Better performance than full virtualization.

→ (3) OS-Level Virtualization (Containers, Docker): share host OS kernel, isolate user space. Lightest weight.

→ (4) Hardware-Assisted Virtualization: uses CPU extensions (Intel VT-x, AMD-V) to reduce overhead.

■ *The syllabus case studies are Xen (paravirtual), VMware (full virt), Microsoft Hyper-V.*

Q13

★ IMPORTANT

What is a Hypervisor? What are Type 1 and Type 2 Hypervisors?

→ A Hypervisor (VMM — Virtual Machine Monitor) is software that creates and manages virtual machines.

→ Type 1 (Bare-Metal): runs directly on hardware without host OS. Examples: VMware ESXi, Microsoft Hyper-V, Xen, KVM. More performant and secure — used in data centers.

→ Type 2 (Hosted): runs on top of a host OS. Examples: VMware Workstation, VirtualBox, Parallels. Easier to set up — used on developer workstations.

→ Key functions: CPU scheduling among VMs, memory management (shadow page tables), I/O virtualization, VM isolation.

Q14

★ IMPORTANT

♦ PRACTICAL

What is KVM? How is it installed and configured? (Lab Assignment)

→ KVM (Kernel-based Virtual Machine) is a full virtualization solution for Linux using hardware virtualization extensions (Intel VT-x/AMD-V). KVM is a Type 1 hypervisor built into the Linux kernel.

→ Installation:

```
# Check hardware support
```

```
egrep -c '(vmx|svm)' /proc/cpuinfo # >0 means supported
```

```
sudo apt install qemu-kvm libvirt-daemon-system virtinst
```

```
sudo systemctl enable --now libvirtd
```

```
sudo usermod -aG libvirt,kvm $USER
```

```
virt-install --name=vm1 --ram=1024 --vcpus=1 \
```

```
--disk path=/var/lib/libvirt/images/vm1.img,size=10 \
```

```
--cdrom=/path/to/ubuntu.iso
```

→ Management: virsh (CLI), virt-manager (GUI).

■ *KVM lab outcome: students learn to create, start, stop, clone VMs.*

Q15**What is Virtual Clustering?**

→ Virtual Clustering creates a cluster of virtual machines running on physical hardware, providing the same capabilities as a physical cluster with added flexibility. Benefits: Easy to add/remove nodes (scale in/out), snapshot and restore cluster state, move VMs between hosts (live migration), test cluster configurations without physical hardware.

→ Implementation: Cluster nodes are VMs managed by Pacemaker/Corosync. Shared storage via NFS or SAN.

→ Use case: High-availability web servers, distributed databases (Cassandra, MongoDB), HPC workloads.

Q16**★ IMPORTANT****What is CPU, Network, and Storage Virtualization?**

→ CPU Virtualization: multiple virtual CPUs (vCPUs) mapped to physical CPUs. Scheduler allocates time slices. Overcommit: more vCPUs than physical CPUs — works if VMs don't all run at 100% simultaneously.

→ Network Virtualization: virtual switches (vSwitch, OVS), VLANs, virtual NICs (vNIC). SDN (Software Defined Networking) decouples control plane from data plane. VXLAN extends VLANs across data centers.

→ Storage Virtualization: logical storage volumes mapped to physical storage. LVM (Logical Volume Manager) on Linux, thin provisioning, storage tiering. SAN virtualization (IBM SVC, EMC VPLEX) pools multiple SANs.

UNIT IV · CLOUD PLATFORMS & APPLICATIONS

07 Hours

Course Outcomes: CO4

Q17**★ IMPORTANT****What are the key components of Amazon Web Services (AWS)?**

→ Compute: EC2 (virtual servers), Lambda (serverless), ECS/EKS (containers).

→ Storage: S3 (object), EBS (block), EFS (file), Glacier (archival).

→ Database: RDS (relational — MySQL/PostgreSQL/Oracle), DynamoDB (NoSQL key-value), ElastiCache (Redis/Memcached).

→ Networking: VPC (virtual private cloud), Route 53 (DNS), CloudFront (CDN), ELB (load balancer).

→ AI/ML: SageMaker, Rekognition, Comprehend, Polly.

→ Management: CloudWatch (monitoring), CloudFormation (IaC), IAM (identity & access management).

■ AWS is the market leader with ~33% cloud market share.

Q18**★ IMPORTANT****What is Amazon EC2? Explain its key concepts.**

→ EC2 (Elastic Compute Cloud) provides resizable virtual servers (instances) in AWS.

→ Key concepts:

→ Instance Types: general purpose (t3, m5), compute optimized (c5), memory optimized (r5), GPU (p3, g4).

→ AMI (Amazon Machine Image): template for instance (OS + software).

→ Security Groups: virtual firewall controlling inbound/outbound traffic.

→ Key Pairs: SSH authentication for Linux instances.

→ Elastic IP: static public IP address.

→ Pricing: On-demand (pay per second), Reserved (1-3 year commitment, up to 75% discount), Spot (unused capacity, up to 90% discount, can be interrupted).

■ EC2 is the case study option for Lab Assignment 1.

Q19**★ IMPORTANT**

What is Amazon DynamoDB?

→ DynamoDB is AWS's fully managed NoSQL key-value and document database designed for single-digit millisecond performance at any scale.

→ Key features: Serverless (no cluster to manage), auto-scaling, multi-region replication (Global Tables), ACID transactions, DynamoDB Streams (change data capture), DAX (in-memory cache).

→ Data model: Table with items (rows) and attributes (columns). Primary Key: Partition Key (mandatory) + optional Sort Key. Secondary Indexes for alternate query patterns.

→ Vs. SQL: no fixed schema, no joins, denormalization required. Used by: Amazon.com, Lyft, Airbnb, Samsung.

Q20**★ IMPORTANT**

What is Microsoft Azure? What are its core services?

→ Azure is Microsoft's cloud platform with 200+ services across 60+ regions.

→ Core services:

→ Compute: Azure VMs, Azure Functions (serverless), AKS (Kubernetes Service), App Service.

→ Storage: Blob Storage (object), Azure Files (SMB), Azure Disk (block).

→ Database: Azure SQL (managed SQL Server), Cosmos DB (NoSQL, multi-model), Azure Database for PostgreSQL/MySQL.

→ Networking: Virtual Network, Azure Load Balancer, Application Gateway, Azure CDN.

→ AI: Azure OpenAI Service, Cognitive Services, Azure ML.

→ DevOps: Azure DevOps, GitHub Actions integration.

■ *Azure SQL is the PaaS managed database — the Exam and Lab emphasize SQL Azure.*

Q21

★ IMPORTANT

✦ PRACTICAL

What is Google App Engine? How is it installed and configured? (Lab Assignment 2)

→ Google App Engine (GAE) is a PaaS for deploying web applications without managing servers — fully managed runtime.

→ Installation:

```
# Install Google Cloud SDK
```

```
curl https://sdk.cloud.google.com | bash
```

```
gcloud init
```

```
gcloud app create --region=asia-south1
```

```
# Create app.yaml
```

```
runtime: python39
```

```
handlers:
```

```
- url: /*.*
```

```
script: auto
```

```
# Deploy
```

```
gcloud app deploy
```

```
gcloud app browse
```

→ Key features: Autoscaling (scale to zero), versioning/traffic splitting, supports Python, Java, Node.js, Go, PHP, Ruby.

■ *Free tier: 28 instance-hours/day. Used by Snapchat, Khan Academy.*

Q22

★ IMPORTANT

✦ PRACTICAL

What is Salesforce and how is an application created using Apex? (Lab Assignment 3&4)

→ Salesforce is a cloud CRM (Customer Relationship Management) platform. Apex is Salesforce's proprietary strongly-typed OOP language similar to Java, running on Salesforce servers.

→ Creating an App:

// Apex Class example

```
public class ContactManager {  
    public static void createContact(String name, String email) {  
        Contact c = new Contact();  
        c.LastName = name;  
        c.Email = email;  
        insert c;  
    }  
}
```

→ Key objects: Account, Contact, Opportunity, Lead, Case. SOQL: SELECT Id, Name FROM Contact WHERE Email != null.

→ Flow: Setup → App Manager → New Lightning App → configure tabs, profiles → Apex classes for logic.

■ *Salesforce uses Force.com platform — multi-tenant architecture where all customers share infrastructure.*

Q23**What is OpenStack? Explain its architecture.**

→ OpenStack is an open-source cloud computing platform for building public and private clouds.

→ Key components:

→ Nova: compute service (manages VMs).

→ Swift: object storage.

→ Cinder: block storage.

→ Neutron: networking (SDN, VLANs).

→ Glance: image service (VM images).

→ Keystone: identity service (authentication, authorization).

→ Horizon: web dashboard.

→ Heat: orchestration (CloudFormation equivalent).

→ Ceilometer: telemetry/metering.

→ Deployment: OpenStack runs on bare-metal servers — used by Walmart, Bloomberg, CERN. Deployed via DevStack (dev), PackStack (RHEL), or Juju (Ubuntu).

Q24**★ IMPORTANT**

What are the key Security Issues in Cloud Computing?

→ Data Security: unauthorized access, data breaches, data loss. Mitigation: encryption (AES-256 at rest, TLS in transit), key management (HSM, AWS KMS).

→ Identity & Access: weak credentials, privilege escalation. Mitigation: MFA, IAM policies, zero-trust architecture.

→ Network Security: DDoS attacks, man-in-the-middle. Mitigation: WAF, VPC, security groups, CloudFront.

→ Compliance: GDPR, HIPAA, PCI-DSS — data must stay in certain regions.

→ Insider Threats: malicious or negligent cloud provider employees.

→ Shared Responsibility: provider secures infrastructure; customer secures data and applications.

■ *Acunetix is the cloud security tool mentioned in the syllabus — web vulnerability scanner.*

Q25**★ IMPORTANT**

What is the Shared Responsibility Model in Cloud Security?

→ The Shared Responsibility Model divides security duties:

→ Cloud Provider is responsible for: Physical security of data centers, network infrastructure, hypervisor security, host operating system.

→ Customer is responsible for: Data encryption, access management (IAM), OS patching (IaaS), application security, network configuration (security groups, VPCs).

→ It varies by service model: IaaS — customer more responsible (OS upward). PaaS — provider handles runtime and OS. SaaS — provider handles almost everything.

■ *Most breaches in cloud are customer-side (misconfigured S3 buckets, weak passwords) not provider-side.*

Q26**★ IMPORTANT**

What is CIA Triad in cloud security?

→ CIA Triad is the foundation of information security:

→ Confidentiality: ensure only authorized users can access data. Mechanisms: encryption (AES, RSA), access controls (RBAC), VPN, MFA.

→ Integrity: ensure data is not altered without authorization. Mechanisms: digital signatures, checksums (SHA-256), audit logs, version control.

→ Availability: ensure systems and data are accessible when needed. Mechanisms: redundancy, load balancing, backups, auto-scaling, DDoS protection.

■ *A fourth dimension increasingly added is Non-Repudiation — ensuring actions cannot be denied (achieved via digital signatures and audit trails).*

Q27

What are Cloud Risk Types and Risk Management strategies?

→ Types of Cloud Risks:

→ (1) Governance Risks: lack of control over data and services.

→ (2) Compliance Risks: violation of regulations (GDPR, HIPAA).

→ (3) Trust Risks: reliance on cloud provider's security promises.

→ (4) Architecture Risks: poor design (single region deployment).

→ (5) Operational Risks: accidental deletion, misconfiguration.

→ (6) Legal Risks: data sovereignty, contracts.

→ Risk Management Strategies: Risk Avoidance (don't put sensitive data in cloud), Risk Mitigation (encryption, redundancy), Risk Transfer (cyber insurance), Risk Acceptance (for low-impact risks).

→ Enterprise Risk Management (ERM) integrates cloud risks into organization-wide risk framework.

Q28

What is Content-Level Security and Cloud Digital Persona?

→ Content-Level Security: applies security controls based on content type and sensitivity — DRM (Digital Rights Management), DLP (Data Loss Prevention) tools scan content for sensitive data (SSNs, credit card numbers) and prevent unauthorized transmission.

→ Cloud Digital Persona: a user's digital identity across cloud services — includes authentication credentials, behavioral biometrics, device fingerprint, and usage patterns. Used for adaptive authentication: if login behavior deviates from persona baseline (unusual location, time), additional verification required. Identity Federation: single identity across multiple cloud services via SAML, OAuth 2.0, OpenID Connect.

UNIT VI · ADVANCED TECHNIQUES IN CLOUD COMPUTING

07 Hours

Course Outcomes: CO6

Q29**★ IMPORTANT**

What are Containers? What is Docker?

→ Containers are lightweight, portable units that package application code, dependencies, and runtime into an isolated environment sharing the host OS kernel (unlike VMs which include full OS).

→ Docker is the dominant container platform.

Key Docker commands

```
docker pull ubuntu # download image
```

```
docker run -it ubuntu bash # run container
```

```
docker build -t myapp . # build from Dockerfile
```

```
docker ps # list running containers
```

```
docker stop # stop container
```

```
docker push myapp:latest # push to registry
```

→ Advantages over VMs: faster startup (seconds vs minutes), less overhead (MBs vs GBs), higher density, consistent environments (dev/prod parity).

■ *DevOps case studies in syllabus: DocuSign, Forter, Gengo all use containers.*

Q30**★ IMPORTANT**

What is Kubernetes (K8s)?

→ Kubernetes is an open-source container orchestration system for automating deployment, scaling, and management of containerized applications.

→ Key concepts:

→ Pod: smallest unit — one or more containers sharing network and storage.

→ Deployment: manages replicas of pods, rolling updates.

→ Service: stable endpoint for accessing pods (ClusterIP, NodePort, LoadBalancer).

→ Namespace: virtual cluster within K8s cluster.

→ ConfigMap/Secret: configuration and sensitive data management.

→ Helm: package manager for K8s (like apt for Ubuntu).

```
kubectl apply -f deployment.yaml
```

```
kubectl get pods -n production
```

```
kubectl scale deployment myapp --replicas=5
```

■ *Kubernetes was created by Google, open-sourced in 2014.*

Q31**★ IMPORTANT**

What is DevOps? How does it relate to Cloud?

→ DevOps is a cultural and technical movement combining Development and Operations teams to deliver software faster and more reliably. Key practices:

→ CI/CD (Continuous Integration/Continuous Deployment): automated build, test, deploy pipeline.

→ Infrastructure as Code (IaC): define infrastructure in code (Terraform, CloudFormation).

→ Monitoring and Logging: centralized observability (ELK Stack, Prometheus, Grafana).

→ Cloud enables DevOps: on-demand environments (no waiting for hardware), managed services reduce operational burden, auto-scaling handles load spikes.

■ *Case studies: DocuSign — 24-hour release cycles became 2-hour cycles with DevOps; Forter — real-time fraud detection improved with DevOps pipelines.*

Q32

What is Mobile Cloud Computing?

→ Mobile Cloud Computing (MCC) offloads compute-intensive tasks from mobile devices to the cloud, overcoming mobile limitations (battery, compute, storage).

→ Architecture: Mobile app sends task to cloud backend via REST API. Cloud processes and returns result. Mobile displays result.

→ Challenges: Network latency (critical for real-time apps), intermittent connectivity, security (data in transit), context-awareness (location, user state).

→ Edge Computing extension: process closer to mobile device at edge servers (CDN edge nodes, telco edge) to reduce latency — used for AR/VR, autonomous vehicles, real-time gaming.

Q33**★ IMPORTANT**

What is Edge Computing vs Cloud Computing?

→ Cloud Computing: centralized data centers (may be 1000s of km away). High latency (50-200ms). Ideal for batch processing, analytics, storage, AI training.

→ Edge Computing: compute at or near data source (factory floor, retail store, base station). Ultra-low latency (<5ms). Ideal for real-time IoT processing, autonomous vehicles, smart manufacturing.

→ Fog Computing: intermediate layer between edge and cloud — extends cloud closer to edge.

→ Distributed Cloud Computing: cloud provider deploys infrastructure at customer edge locations — AWS Outposts, Azure Stack Edge, Google Distributed Cloud.

■ *5G enables edge computing by providing high bandwidth, low latency connectivity.*

Q34**★ IMPORTANT****What is IoT and Cloud Convergence?**

→ IoT (Internet of Things) connects physical devices (sensors, actuators) to the internet. Cloud provides the scalable backend for IoT:

→ IoT + Cloud Architecture: Devices → IoT Gateway → Cloud IoT Platform → Analytics → Applications.

→ AWS IoT Core: managed service for IoT device connectivity, security, and message routing.

→ Azure IoT Hub: bidirectional communication between IoT devices and cloud.

→ Usage examples: Smart Home: Alexa/Google Home — voice commands processed in cloud, control home devices. Automobile: Tesla — over-the-air updates via cloud, real-time telemetry analysis. Healthcare: patient monitoring devices stream vitals to cloud for real-time alerts.

■ 75 billion IoT devices expected by 2025 globally — cloud is the backbone.

Q35**★ IMPORTANT****♦ PRACTICAL****Explain the Cloud SaaS Mini Project (Lab Assignment 5).**

→ Goal: Setup your own cloud for SaaS over existing LAN — implement a basic cloud controller with HDFS-like functionality.

→ Requirements: Divide files into blocks/segments, upload/download encrypted.

```
import os, json, hashlib
from Crypto.Cipher import AES
BLOCK_SIZE = 1024*1024 # 1MB blocks
def split_and_encrypt(filepath, key):
    blocks = []
    cipher = AES.new(key, AES.MODE_EAX)
    with open(filepath, 'rb') as f:
        i = 0
        while chunk := f.read(BLOCK_SIZE):
            enc, tag = cipher.encrypt_and_digest(chunk)
            block_name = f'block_{i}.enc'
            with open(block_name, 'wb') as bf: bf.write(enc)
            blocks.append({'id':i, 'file':block_name, 'hash':hashlib.md5(chunk).hexdigest()})
            i += 1
    return blocks
```

→ Components: Controller (tracks block locations), DataNodes (LAN machines storing blocks), Client (splits, encrypts, uploads; downloads, decrypts, reassembles).

→ Output: File reconstructed correctly after download; encryption verified with AES key.

PART II — LAB PRACTICAL QUESTIONS

All Lab Assignments · Azure · EC2 · App Engine · KVM · Salesforce · Mini Project

Cloud Computing Lab Assignments (All Compulsory)

Q1

★ IMPORTANT

↗ PRACTICAL

Case Study — Microsoft Azure (Lab Assignment 1 Option A)

→ Microsoft Azure is a cloud computing platform by Microsoft with 200+ services across 60+ global regions.

→ Key features studied: Azure Portal (web console for resource management), Azure Resource Manager (ARM) — infrastructure deployment model, Resource Groups (logical containers for related resources), Azure Active Directory (cloud identity provider, MFA, SSO).

→ Core services: Azure VMs (IaaS), Azure App Service (PaaS web hosting), Azure SQL (managed relational DB), Azure Blob Storage, Azure Functions (serverless), Azure DevOps.

→ Pricing: Pay-as-you-go; Free Tier: 12 months free services + always-free tier (5GB blob, 250GB SQL).

→ Architecture Advantage: Hybrid cloud (Azure Arc), deep integration with Windows Server, Active Directory, and Microsoft 365 — ideal for enterprises.

Q2

★ IMPORTANT

↗ PRACTICAL

Case Study — Amazon EC2 (Lab Assignment 1 Option B)

→ Amazon EC2 provides resizable virtual compute capacity in AWS.

→ Key concepts: Instance Types: t3.micro (free tier), m5.large (general), c5.2xlarge (compute). AMI: Amazon Machine Image — template with OS and pre-installed software. Security Groups: stateful firewall — inbound/outbound rules by port and IP. Key Pairs: RSA key for SSH access. Elastic IP: static public IP. Auto Scaling Groups: automatically scale instance count based on CPU/memory metrics.

→ Launch an EC2 instance:

Via AWS CLI

```
aws ec2 run-instances \
```

```
--image-id ami-0abcdef1234567890 \
```

```
--instance-type t2.micro \
```

```
--key-name MyKeyPair \
```

```
--security-group-ids sg-12345678
```

→ Use cases: Web hosting, batch processing, HPC, game servers.

Q3

★ IMPORTANT

✦ PRACTICAL

Salesforce Application Creation using Apex (Lab Assignment 3)

→ Apex is Salesforce's proprietary Java-like language executed on Force.com servers.

→ Key concepts:

→ SOQL (Salesforce Object Query Language): `SELECT Id, Name, Email FROM Contact WHERE AccountId != null`.

→ DML Operations: insert, update, delete, upsert.

→ Triggers: before/after insert/update/delete on objects.

```
// Trigger example: auto-set contact title
trigger ContactTrigger on Contact (before insert) {
    for(Contact c : Trigger.new) {
        if(c.Title == null) c.Title = 'Customer';
    }
}
```

→ Visualforce: HTML-like UI framework for custom pages.

→ Lightning Components: modern React-like UI framework (Aura/LWC).

→ Deployment: Sandbox (test environment) → Production via Change Sets or CI/CD.

→ Output: A custom Salesforce app with Account, Contact, Opportunity objects and custom Apex logic.

Q4

★ IMPORTANT

✦ PRACTICAL

Mini Project — Custom Salesforce Application (Lab Assignment 4)

→ Goal: Build a complete custom cloud application on Salesforce platform.

→ Design steps: (1) Define custom objects (e.g., Student__c, Course__c, Enrollment__c). (2) Create fields, relationships, validation rules. (3) Write Apex classes for business logic. (4) Create Visualforce/LWC pages for UI. (5) Set up reports and dashboards.

```
// Custom Object Query
List students = [SELECT Id, Name, Course__r.Name
FROM Enrollment__c
WHERE Course__r.Name = 'Cloud Computing'];

// Process records
for(Student__c s : students) System.debug(s.Name);
```

→ Testing: Apex Test Classes (must achieve 75%+ code coverage for deployment).

→ Output: Functional cloud application with CRUD operations, business logic, and UI deployed on Salesforce cloud.

Q5**★ IMPORTANT****✦ PRACTICAL**

Mini Project — SaaS Cloud Setup over LAN (Lab Assignment 5)

→ Goal: Implement basic cloud controller using open-source technologies with HDFS-like operations.

→ Architecture: NameNode (controller tracking file blocks), DataNodes (LAN machines storing blocks), Client (upload/download interface).

→ Key Operations:

→ (1) File Splitting: divide file into fixed-size blocks (e.g., 1MB).

→ (2) Encryption: AES-256 encrypt each block before storage.

→ (3) Metadata: store block manifest (block IDs, DataNode IPs, checksums) in JSON.

→ (4) Upload: send encrypted blocks to different DataNodes via socket/HTTP.

→ (5) Download: fetch blocks from DataNodes, decrypt, reassemble.

Block reassembly

```
def reassemble(manifest, key):
```

```
    with open(manifest['filename'], 'wb') as out:
```

```
        for block in sorted(manifest['blocks'], key=lambda x:x['id']):
```

```
            data = download_block(block) # fetch from DataNode
```

```
            out.write(decrypt(data, key)) # AES decrypt
```

→ Complexity: Upload $O(n)$ blocks, Download $O(n)$ blocks where $n = \text{file_size}/\text{block_size}$.

→ Output: File successfully uploaded in encrypted chunks and reconstructed identically on download. Hash verification ensures integrity.